

2주차 · ROS 2 기초 — 노드와 토픽

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

- **과목:** 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용:** ① ROS 2 설치 ② 두 프로그램이 서로 데이터를 주고받게 만들기

★ 시작 전 확인

- 1주차 WSL 설치가 끝나 있어야 함
- `xeyes` 가 안 뜨는 학생은 **지금 손 들 것**. 이번 주차 설치를 해도 3주차에 막힘
- 저장공간 **20 GB 이상** 여유 확인: `df -h` 실행 후 `/` 행의 Avail 값 확인

이 주차를 마치면 할 수 있어야 하는 것

1. ROS 2가 왜 필요한지 설명
2. **노드 · 토픽 · 메시지**가 각각 무엇인지 설명
3. `ros2` 명령어로 실행 중인 노드와 토픽을 조사
4. **내 손으로 노드를 작성**해서 데이터를 주고받기
5. **QoS 불일치**를 재현하고 원인을 진단
6. **표본화 주기**가 왜 중요한지 설명

준비물

항목	내용
환경	1주차에 만든 WSL2 + Ubuntu 22.04 (<code>wsl -l -v</code> 로 확인)
저장공간	20 GB 이상 — ROS 2 설치에 필요
터미널	<code>terminator</code> (1주차 2-6절). 창을 두 개 이상 띄워 쓴다
인터넷	패키지 내려받기. 학교 와이파이면 시간이 더 걸린다

⚠ 1주차 설치를 완료하지 못한 경우

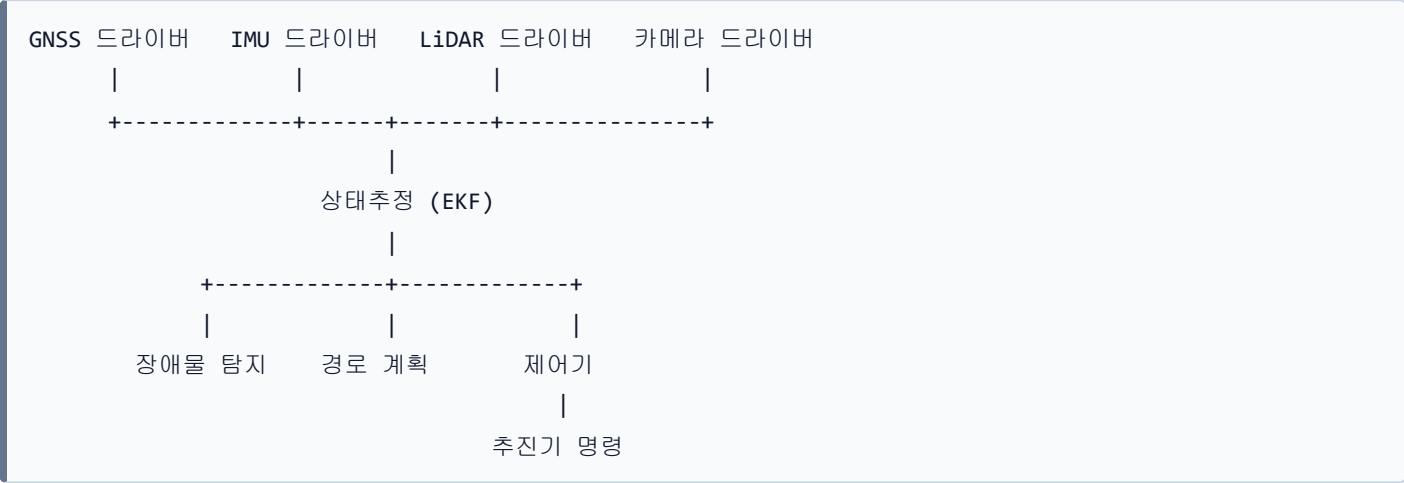
이번 주차 실습은 **전부 WSL 안에서** 이뤄진다. 설치부터 하면 따라올 수 없다.
수업 시작 전에 조교에게 알릴 것.

1부 · 이론

1-1. ROS 2는 무엇을 해결하는가

문제 상황

- 자율운항 보트 한 대에 붙는 프로그램들



- 이것을 하나의 큰 프로그램으로 만들면 생기는 문제

문제	결과
모듈이 서로 얽힘	LiDAR 처리가 느려지면 제어기까지 멈춤
부분 수정 불가	카메라 알고리즘 하나 바꾸려고 전체 재컴파일
언어 고정	제어는 MATLAB, 인지는 Python으로 짜고 싶은데 방법 없음
장애 전파	한 곳이 죽으면 배 전체가 죽음

ROS 2의 해법

- 프로그램을 독립된 조각(노드)으로 쪼개고, 그 사이 통신을 표준화

문제	ROS 2의 해법
모듈 결합	노드를 별도 프로세스로 분리
언어 다양성	C++ · Python · MATLAB 이 같은 토픽으로 대화
데이터 형식	표준 메시지 타입 (<code>sensor_msgs/Imu</code> 등)
분산 실행	같은 네트워크의 다른 컴퓨터와도 자동 연결
재현성	<code>rosvbag2</code> 로 모든 데이터를 기록·재생

i 본 과목에서 결정적인 지점

6주차에 **Windows**의 **Simulink**와 **WSL**의 **Gazebo**가 서로 대화하게 됨.
그것을 가능하게 하는 것이 바로 이 구조임.

ROS 1과 ROS 2

항목	ROS 1	ROS 2
통신	중앙 관리자(roscore) 필요	관리자 없이 서로 자동 발견
실시간성	약함	지원
통신 품질(QoS)	없음	선택 가능
상태	지원 종료	현재 표준

- 본 과목에서 사용할 것: **ROS 2 Humble Hawksbill** (장기 지원 버전, 2027년 5월까지)

1-2. 노드와 토픽



- 핵심 1 발행자는 누가 듣는지 모른다. 구독자도 누가 보내는지 모른다
- 핵심 2 한 토픽을 여러 노드가 동시에 구독할 수 있다
- 핵심 3 토픽 이름과 메시지 타입과 QoS 가 모두 맞아야 연결된다

용어

용어	뜻	비유
노드 (Node)	하나의 일을 하는 독립 프로그램	직원 한 명
토픽 (Topic)	데이터가 흐르는 이름 있는 통로	사내 게시판
발행 (Publish)	토픽에 데이터를 올림	게시판에 글 쓰기
구독 (Subscribe)	토픽에서 데이터를 받음	게시판 구독
메시지 (Message)	데이터의 형식	정해진 양식의 문서

핵심 성질 3가지

- 발행자는 누가 듣는지 모름. 구독자도 누가 보내는지 모름
- 한 토픽을 여러 노드가 동시에 구독 가능
- 토픽 이름 + 메시지 타입 + QoS 가 전부 맞아야 연결됨

토픽 외의 통신 방식 (참고)

방식	쓰임	예
토픽	계속 흐르는 데이터	센서 값 ← 본 과목의 90%
서비스	1회성 요청-응답	"지금 상태를 초기화해줘"
액션	오래 걸리고 취소 가능한 작업	"저 지점까지 가라"
파라미터	노드의 설정값	PID 게인

1-3. 메시지 타입

자주 쓰는 것

타입	내용	VRX에서
std_msgs/Float64	실수 하나	추력 명령
sensor_msgs/NavSatFix	위도 · 경도 · 고도	GPS
sensor_msgs/Imu	자세 · 각속도 · 가속도	IMU
sensor_msgs/PointCloud2	3D 점 덩어리	LiDAR
sensor_msgs/LaserScan	2D 거리 배열	LiDAR 축약
sensor_msgs/Image	영상	카메라
nav_msgs/Odometry	위치 + 자세 + 속도	상태추정 결과

- 구조를 확인하는 명령어

```
ros2 interface show sensor_msgs/msg/Imu
```

Header — 모든 센서 메시지의 앞부분

```
std_msgs/Header header
  builtin_interfaces/Time stamp      # 이 데이터를 측정한 시각
  string frame_id                   # 어느 좌표계 기준인가
```

⚠ stamp 와 frame_id 는 필수 항목이다

- stamp 가 없으면 GPS(10 Hz)와 IMU(100 Hz)를 융합할 수 없음.
어느 시점끼리 짝지어야 하는지 알 수 없기 때문
- frame_id 가 없으면 LiDAR가 본 장애물이 배 기준인지 지구 기준인지 알 수 없음
- 3주차 TF2에서 다시 나옴

1-4. DDS 와 Domain ID

문제 상황

- ROS 2에는 중앙 관리자가 없음 → 노드들이 네트워크에서 서로를 자동으로 찾음
- 실습실에서 20명이 동시에 작업하면 → 모두의 노드가 서로 보임
- 결과: 옆 사람의 추력 명령이 내 배를 움직임

해결 — ROS_DOMAIN_ID

- 같은 번호를 가진 노드끼리만 통신
- 범위: 0 ~ 101

★ 본 과목의 규칙

팀 번호를 Domain ID로 사용. 1팀 → 1, 2팀 → 2 ...

- 설정 방법 (한 번만 하면 됨)

```
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
source ~/.bashrc
```

- 확인

```
echo $ROS_DOMAIN_ID
```

- 정상 출력: 7 (자기 팀 번호)

! 6주차 복선

MATLAB과 연동할 때 양쪽 Domain ID가 같아야 통신됨. 지금 기억해 둘 것.

1-5. QoS — 통신 품질 정책

무엇인가

- "이 데이터를 어떻게 전달할지" 선택하는 설정
- ROS 1에는 없던 개념

두 가지 핵심 항목

① Reliability (신뢰성)

정책	동작	적합한 데이터
RELIABLE	도착 보장. 실패하면 재전송	명령, 목표점
BEST_EFFORT	빠르게 보내고 잊음. 유실 허용	고빈도 센서 (카메라, LiDAR)

② Durability (지속성)

정책	동작
VOLATILE	늦게 들어온 구독자는 과거 데이터를 못 받음
TRANSIENT_LOCAL	늦게 들어와도 마지막 값을 받음 (지도, 정적 TF)

QoS 불일치 — 본 과목에서 가장 혼동하기 쉬운 부분

발행자	구독자	결과
RELIABLE	RELIABLE	연결
RELIABLE	BEST_EFFORT	연결
BEST_EFFORT	RELIABLE	연결 안 됨

- 규칙: 구독자의 요구가 발행자가 제공하는 것보다 엄격하면 실패

✕ 오류 메시지 없이 잘못된 결과가 나온다

- `ros2 topic list` → 토픽이 보임
- `ros2 topic hz` → 데이터가 흐른다고 나옴
- 내 노드만 콜백이 한 번도 안 불림
- 학생은 자기 콜백 코드를 의심하며 한 시간을 씀

- 진단 명령어 (한 줄)

```
ros2 topic info /토픽이름 --verbose
```

- 이번 주차 실습에서 일부러 재현해 봄

1-6. 표본화 — 왜 신호처리를 알아야 하는가

센서는 연속 신호를 잘라서 준다

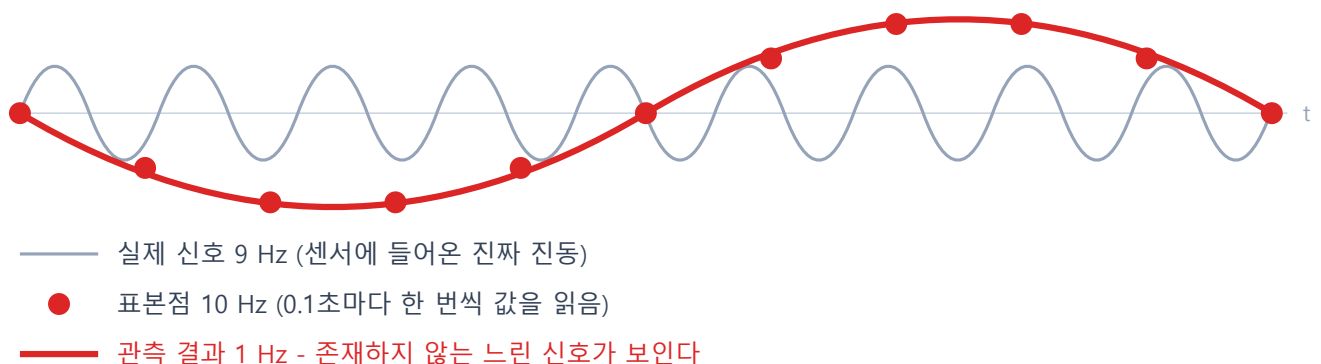
센서	주기	VRX
IMU	100~200 Hz	100 Hz
GNSS	1~10 Hz	10 Hz
LiDAR	10~20 Hz	10 Hz
카메라	15~30 Hz	20 Hz
제어기	10~100 Hz	100 Hz

표본화 정리 (Nyquist)

- 최대 주파수 `f_max` 인 신호를 잃지 않으려면
- 표본화 주파수가 `f_s > 2 × f_max` 여야 함

에일리어싱 — 조건을 어기면

에일리어싱 - 9 Hz 신호를 10 Hz 로 표본화하면 1 Hz 로 둔갑한다



- 위 그림 읽는 법

요소	의미
회색 촘촘한 파형	실제 신호 9 Hz
빨간 점	0.1초마다 읽은 값 (10 Hz 표본화)
빨간 굵은 곡선	본 과목에서 관측하게 되는 신호 — 1 Hz

- 결론: 9 Hz 신호를 10 Hz로 읽으면 **존재하지 않는 1 Hz** 신호가 보임

배에서 실제로 일어나는 일

1. 파랑에 의한 선체 진동이 IMU에 실려 옴
2. 표본화 주기가 안 맞으면 **없는 저주파 흔들림**이 관측됨
3. 제어기가 그것을 쫓아가려 함
4. 배가 진동함

대책

- 표본화 **전에** 저역통과 필터로 고주파를 자름
- 또는 충분히 빠르게 표본화한 뒤 디지털 필터를 거쳐 다운샘플링
- → 이번 주차 과제가 이것

지연 (Latency)

물리량 발생 → 센서 측정 → 드라이버 → 전송 → 내 노드 → 처리 → 명령

t=0 t=2ms t=5ms t=8ms t=10ms t=15ms t=16ms

- 제어에서 지연은 **위상 지연**이 되어 시스템을 불안정하게 만듦
- 그래서 `header.stamp` 는 수신 시각이 아니라 **측정 시각**으로 채워야 함

2부 · 실습

2-1. ROS 2 Humble 설치

i 소요 시간 약 20~40분

다운로드가 큼. 중간에 노트북을 절전 모드로 두지 말 것.

1단계 — 언어 설정

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

- 확인

```
locale
```

- 정상 출력에 `LANG=en_US.UTF-8` 이 보이면 됨

2단계 — ROS 저장소 등록

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
```

- 중간에 `Press [ENTER] to continue` 가 나오면 `Enter`

```
sudo apt update && sudo apt install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] ht
tp://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/
apt/sources.list.d/ros2.list > /dev/null
```

⚠ 위 명령은 한 줄이다

문서에서 복사할 때 줄이 나뉘지 않도록 주의. 회색 상자를 통째로 복사할 것.

3단계 — 설치

```
sudo apt update
sudo apt upgrade -y
```

```
sudo apt install -y ros-humble-desktop
```

```
sudo apt install -y python3-colcon-common-extensions python3-rosdep
```

4단계 — 자동 적용 설정

- 매번 손으로 설정하지 않도록 `.bashrc` 에 등록

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
source ~/.bashrc
```

★ 7 을 자기 팀 번호로 바꿀 것

5단계 — rosdep 초기화

```
sudo rosdep init
rosdep update
```

- `sudo rosdep init` 에서 "already exists" 오류가 나면 무시하고 진행

2-2. 설치 확인 — 첫 통신

실행

1. `terminator` 실행
2. `Ctrl + Shift + E` 로 좌우 분할

왼쪽 칸

```
ros2 run demo_nodes_cpp talker
```

- 정상 출력

```
[INFO] [1789000000.123456789] [talker]: Publishing: 'Hello World: 1'
[INFO] [1789000001.123456789] [talker]: Publishing: 'Hello World: 2'
```

오른쪽 칸

```
ros2 run demo_nodes_py listener
```

- 정상 출력

```
[INFO] [1789000001.223456789] [listener]: I heard: [Hello World: 1]
[INFO] [1789000002.223456789] [listener]: I heard: [Hello World: 2]
```

위 과정에서 일어난 일

- C++로 짠 노드와 Python으로 짠 노드가
- 서로의 존재를 모르는 채로
- 자동으로 찾아서 대화했음
- 이것이 ROS 2의 핵심임

- 종료: 각 칸에서 `Ctrl + C`

2-3. 조사 명령어 익히기

- `talker` 를 켜 둔 채로 새 분할에서 실행

노드 조사

```
ros2 node list
```

```
/talker
```

```
ros2 node info /talker
```

- 이 노드가 무엇을 발행,구독하는지 나옴

토픽 조사

```
ros2 topic list
```

```
ros2 topic list -t
```

- `-t` 를 붙이면 메시지 타입까지 표시

```
ros2 topic echo /chatter
```

- 실제 데이터가 흘러나옴. 종료는 `Ctrl + C`

```
ros2 topic hz /chatter
```

- 발행 주기 측정. 가장 자주 쓰는 명령어

```
average rate: 1.000  
min: 0.999s max: 1.001s std dev: 0.00050s window: 10
```

```
ros2 topic info /chatter --verbose
```

- QoS 를 포함한 상세 정보. 문제 진단의 핵심

메시지 구조 확인

```
ros2 interface show std_msgs/msg/String
```

명령줄에서 직접 발행

```
ros2 topic pub /chatter std_msgs/msg/String "{data: 'from CLI'}" -r 1
```

노드 그래프 그림으로 보기

```
rqt_graph
```

- 노드(타원)와 토픽(사각형)의 연결이 그림으로 나옴
- 시스템이 복잡해지면 이것이 유일한 디버깅 수단이 됨

데이터 기록 · 재생

```
ros2 bag record /chatter
```

```
ros2 bag play rosbag2_2026_09_10-14_30_00
```

i 왜 중요한가

11주차에 충돌회피 알고리즘을 비교할 때
같은 LiDAR 데이터를 반복 재생해서 알고리즘만 바꿔 비교함.
공정한 비교의 필수 도구임.

2-4. 내 패키지 만들기

워크스페이스 생성

```
mkdir -p ~/capstone_ws/src  
cd ~/capstone_ws/src
```

패키지 생성

```
ros2 pkg create --build-type ament_python --license Apache-2.0 usv_basics
```

- 생성된 구조

```
usv_basics/  
├─ package.xml           패키지 정보, 의존성  
├─ setup.py              빌드 설정, 실행파일 등록  
├─ setup.cfg  
├─ resource/usv_basics  
└─ usv_basics/           <- 여기에 .py 파일을 넣는다  
    └─ __init__.py
```

빌드

```
cd ~/capstone_ws  
colcon build --symlink-install  
source install/setup.bash
```

--symlink-install 사용을 권장한다

Python 파일이 링크로 연결되어 코드를 고칠 때마다 다시 빌드할 필요가 없음.

- 자동 적용 등록

```
echo "source ~/capstone_ws/install/setup.bash" >> ~/.bashrc
```

2-5. 첫 노드 작성

발행자

- 파일 위치: `~/capstone_ws/src/usv_basics/usv_basics/simple_talker.py`
- 만드는 방법

```
cd ~/capstone_ws/src/usv_basics/usv_basics
nano simple_talker.py
```

- 아래 내용을 붙여넣기 (`Ctrl + Shift + V`)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleTalker(Node):
    def __init__(self):
        super().__init__('simple_talker')      # 노드 이름
        self.pub = self.create_publisher(      # 발행자 생성
            String, 'usv_chatter', 10)        # 타입, 토픽명, 큐 크기
        self.timer = self.create_timer(0.5, self.on_timer)  # 0.5초마다 = 2 Hz
        self.count = 0

    def on_timer(self):
        msg = String()
        msg.data = f'USV alive: {self.count}'
        self.pub.publish(msg)
        self.get_logger().info(f'published: {msg.data}')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    node = SimpleTalker()
    try:
        rclpy.spin(node)          # 계속 돌면서 타이머 콜백 처리
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

- 저장: `Ctrl + O` → `Enter` → 종료: `Ctrl + X`

구독자

```
nano simple_listener.py
```

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleListener(Node):
    def __init__(self):
        super().__init__('simple_listener')
        self.sub = self.create_subscription(      # 구독자 생성
            String, 'usv_chatter', self.on_msg, 10)

    def on_msg(self, msg):                        # 데이터가 올 때마다 호출됨
        self.get_logger().info(f'received: {msg.data}')

def main(args=None):
    rclpy.init(args=args)
    node = SimpleListener()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

실행파일 등록

```
cd ~/capstone_ws/src/usv_basics
nano setup.py
```

- **entry_points** 부분을 아래로 수정

```
entry_points={
    'console_scripts': [
        'simple_talker = usv_basics.simple_talker:main',
        'simple_listener = usv_basics.simple_listener:main',
    ],
},
```

빌드 및 실행

```
cd ~/capstone_ws
colcon build --symlink-install
source install/setup.bash
```

- 정상 출력

```
Starting >>> usv_basics
Finished <<< usv_basics [1.23s]

Summary: 1 package finished [1.45s]
```

- 터미널 A

```
ros2 run usv_basics simple_talker
```

- 터미널 B

```
ros2 run usv_basics simple_listener
```

- B에 `received: USV alive: 0` 이 흐르면 성공

2-6. QoS 불일치 재현 실험

★ 이번 주차 실습에서 가장 중요한 부분

이 유의 사항을 지금 손으로 만들어 봐야, 3주차에 VRX LiDAR에서 만났을 때 스스로 알아챌.

발행자 — BEST_EFFORT

```
cd ~/capstone_ws/src/usv_basics/usv_basics
nano qos_test_pub.py
```

```

import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from std_msgs.msg import String

class QosTestPub(Node):
    def __init__(self):
        super().__init__('qos_test_pub')
        qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,    # <-- 여기가 핵심
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
        )
        self.pub = self.create_publisher(String, 'qos_topic', qos)
        self.create_timer(0.5, self.on_timer)

    def on_timer(self):
        msg = String()
        msg.data = 'best effort message'
        self.pub.publish(msg)
        self.get_logger().info('published')

def main():
    rclpy.init()
    node = QosTestPub()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

```

구독자 — RELIABLE (더 엄격)

```
nano qos_test_sub.py
```

```

import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from std_msgs.msg import String

class QosTestSub(Node):
    def __init__(self):
        super().__init__('qos_test_sub')
        qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,      # <-- 발행자보다 엄격
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
        )
        self.sub = self.create_subscription(
            String, 'qos_topic', self.on_msg, qos)

    def on_msg(self, msg):
        self.get_logger().info(f'received: {msg.data}')

def main():
    rclpy.init()
    node = QosTestSub()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

```

- `setup.py` 의 `entry_points` 에 두 줄 추가

```

'qos_test_pub = usv_basics.qos_test_pub:main',
'qos_test_sub = usv_basics.qos_test_sub:main',

```

- 빌드

```
cd ~/capstone_ws && colcon build --symlink-install && source install/setup.bash
```

관찰 순서

1. 두 노드를 각각 실행

```
ros2 run usv_basics qos_test_pub
```

```
ros2 run usv_basics qos_test_sub
```

- 관찰: 발행자는 계속 `published` 를 찍는데 구독자는 아무것도 안 나옴
- 관찰: **에러 메시지가 하나도 없음** ← 이것이 이 유의 사항이 위험한 이유

2. 진단

```
ros2 topic info /qos_topic --verbose
```

- 출력에서 **Publishers:** 와 **Subscriptions:** 각각의 **Reliability** 를 비교
- 서로 다른 것이 확인됨

3. 수정

- `qos_test_sub.py` 의 `ReliabilityPolicy.RELIABLE` 을 `ReliabilityPolicy.BEST_EFFORT` 로 변경
- 다시 빌드 후 실행 → 정상 수신

⚠ 3주차에서 다시 다룬다

Gazebo의 센서 토픽 상당수가 `BEST_EFFORT` 로 발행됨.

`ros2 topic echo` 로는 보이는데 내 노드만 못 받으면 **가장 먼저 QoS를 의심할 것.**

마무리

이번 주차 요약

순서	한 일	확인 방법
1	ROS 2 Humble 설치	<code>ros2 --help</code>
2	Domain ID 설정	<code>echo \$ROS_DOMAIN_ID</code>
3	talker / listener 통신	<code>I heard: [Hello World: N]</code>
4	조사 명령어 사용	<code>ros2 topic list</code> / <code>hz</code> / <code>info</code>
5	워크스페이스와 패키지 생성	<code>colcon build</code> 성공
6	내 노드 작성 및 통신	<code>received: USV alive: 0</code>
7	QoS 불일치 재현 · 진단 · 해결	<code>ros2 topic info --verbose</code>

수업 진도 체크

★ 다음 주 수업을 따라가기 위한 최소 조건

이론 이해

- ☐ ROS 2가 무엇을 해결하는지 설명할 수 있다
- ☐ 노드 · 토픽 · 발행 · 구독 · 메시지를 각각 설명할 수 있다
- ☐ `header.stamp` 와 `frame_id` 가 왜 필요한지 안다
- ☐ `ROS_DOMAIN_ID` 를 왜 팀별로 나누는지 안다
- ☐ QoS 불일치가 **에러 없이** 실패한다는 것을 안다
- ☐ 에일리어싱이 무엇인지 그림으로 설명할 수 있다

실습 완료

- ☐ `ros2 run demo_nodes_cpp talker` / `demo_nodes_py listener` 통신 성공
- ☐ `ros2 topic list` / `echo` / `hz` / `info --verbose` 를 모두 써 봤다
- ☐ `rqt_graph` 로 노드 그래프를 확인했다
- ☐ `~/capstone_ws` 워크스페이스 생성 및 빌드 성공
- ☐ 직접 만든 `simple_talker` ↔ `simple_listener` 통신 성공
- ☐ QoS 불일치를 재현하고 진단한 뒤 해결했다
- ☐ `~/.bashrc` 에 `ROS_DOMAIN_ID` 를 팀 번호로 설정했다

다음 주 준비

- ☐ 저장공간 20 GB 이상 확보

과제 2 — 가상 IMU 표본화 실험

- 제출 기한: 3주차 수업 전
- 제출: 코드 + 그래프 + 짧은 분석

① 가상 IMU 발행 노드 (imu_sim)

- 토픽 `/sim/imu` , 타입 `sensor_msgs/Imu` , 50 Hz 발행
- `angular_velocity.z` 에 아래 신호를 실을 것

```
w_z(t) = 0.5 * sin(2*pi*0.5*t)    저주파 0.5 Hz (실제 선회 운동)
        + 0.2 * sin(2*pi*9.0*t)    고주파 9 Hz (파랑 진동)
        + n(t)                     가우시안 잡음, 표준편차 0.02
        + 0.01                     상수 바이어스
```

- `header.stamp` 를 현재 시각으로 정확히 채울 것
- `header.frame_id = "imu_link"`

② 재표본화 구독 노드 (imu_resampler)

- `/sim/imu` 를 구독 → 10 Hz 로 `/sim/imu_10hz` 에 재발행
- 두 방식을 모두 구현하고 비교
 - (a) 단순 데시메이션 — 5개 중 1개만 그대로 통과
 - (b) 이동평균 후 데시메이션 — 최근 5개 평균을 낸 뒤 통과

③ 결과 그래프

- 한 그래프에 세 신호를 겹쳐 그릴 것
 - 원본 50 Hz
 - (a) 단순 데시메이션 10 Hz
 - (b) 이동평균 후 10 Hz
- 축 라벨 · 단위 · 범례 필수

④ 분석 (5~10줄)

- 9 Hz 성분을 10 Hz로 표본화하면 몇 Hz로 둔갑하는가? 그래프에서 확인되는가?
- (a)와 (b) 중 어느 쪽이 나은가? 왜 그런가?
- 이 문제가 실제 배의 heading 제어기에서 어떤 증상으로 나타나겠는가?

🔍 힌트

에일리어싱된 주파수 = $|f_{\text{신호}} - k \times f_{\text{표본화}}|$ 중 나이퀴스트 주파수 이하인 값 (k는 정수)

평가 기준

항목	배점
두 노드 정상 동작 (<code>ros2 topic hz</code> 로 주기 확인 가능)	40%
그래프 품질 (축 라벨, 범례, 단위)	20%
분석의 정확성 — 특히 1번	30%
코드 가독성 (함수 분리, 주석)	10%

막혔을 때

증상	원인	해결
<code>ros2: command not found</code>	환경 미적용	<code>source /opt/ros/humble/setup.bash</code>
내 패키지가 <code>ros2 run</code> 에 안 보임	워크스페이스 미적용	<code>source</code> <code>~/capstone_ws/install/setup.bash</code>
코드를 고쳤는데 반영 안 됨	<code>--symlink-install</code> 없이 빌드	옵션 붙여 재빌드
<code>colcon build</code> 에서 <code>setup.py</code> 오류	<code>entry_points</code> 오타	패키지명.파일명:main 형식 확인
옆자리 학생의 토픽이 보임	Domain ID 같음	팀 번호로 변경 후 <code>source ~/.bashrc</code>
토픽은 보이는데 데이터를 못 받음	QoS 불일치	<code>ros2 topic info <토픽> --verbose</code>
<code>rqt_graph</code> 창이 안 뜸	WSLg 문제	1주차 <code>xeyes</code> 확인으로 복귀
<code>sudo rosdep init</code> 이 실패	이미 초기화됨	무시하고 <code>rosdep update</code> 진행

참고 자료

공식 문서 (북마크 권장)

- ROS 2 Humble 문서 — <https://docs.ros.org/en/humble/>
- 초급 튜토리얼 (CLI) — <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html>
- 초급 튜토리얼 (노드 작성) — <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html>
- QoS 설정 상세 — <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html>
- 표준 메시지 정의 — https://github.com/ros2/common_interfaces

볼트 내 문서

- [QoS가-어긋나면-에러없이-끊긴다](#) — 이번 주차 실습의 배경
- [WSL-VRX-환경구축](#) §3 — ROS 2 설치 절차

다음 주 예고

- 3주차 — Gazebo Garden + VRX 설치, 좌표계와 TF2
- 할 일
 - WAM-V 를 시뮬레이션 환경에 배치 — 시드니 레가타 해역에 WAM-V 스폰
 - 직접 조종해서 움직여 봄
 - ENU / NED 좌표계와 쿼터니언 정리
- 준비물
 - 이번 주차에 작성한 ROS 2 환경
 - 저장공간 20 GB 이상 (VRX 빌드에 필요)
 - 빌드에 30~60분 걸리므로 충전기 지참